Módulo 8: DevOps, Plataforma de Desenvolvimento e Engenharia de Entrega (resumo teórico)

Resumo teórico em português para estudo. Cobre o escopo do módulo 8 do edital do TCE-GO (Técnico de Controle Externo, TI): integração, entrega e implantação contínuas (CI/CD); pipelines; automação de build e testes; infraestrutura como código e gerenciamento de configuração; observabilidade e monitoramento; controle de versão distribuído com Git, GitHub e GitLab, estratégias de branching (Git Flow e trunk-based development), pull requests e revisão de código; contêineres com Docker e Docker Compose; orquestração com Kubernetes; ambientes de desenvolvimento, homologação e produção; e automação de pipelines com GitHub Actions, GitLab CI/CD, Jenkins e ferramentas correlatas. Os comandos e as saídas citadas vêm de laboratórios executados de verdade (L1 a L10); o que não foi executado está marcado como "não testado". A FCC tem pouco histórico neste tema, por isso as questões reais de treino vêm também da FGV e do Cebraspe, sempre com a fonte.

1. Cultura DevOps, CALMS, DORA e DevSecOps

- DevOps é um conjunto de práticas e uma cultura que aproxima desenvolvimento (Dev) e operações (Ops) para entregar software com mais frequência, mais qualidade e menos risco. O foco é o fluxo de valor de ponta a ponta: ideia, código, build, teste, liberação, operação e feedback.
- O modelo CALMS resume os pilares: Culture (colaboração e responsabilidade compartilhada), Automation (automatizar build, teste, implantação e infraestrutura), Lean (lotes pequenos, eliminar desperdício), Measurement (medir para decidir) e Sharing (compartilhar conhecimento e ferramentas).
- As quatro métricas DORA medem a entrega: frequência de implantação e tempo de espera das mudanças (velocidade), taxa de falha das mudanças e tempo de recuperação do serviço (estabilidade). Equipes de alto desempenho são rápidas e estáveis ao mesmo tempo; velocidade e estabilidade não são uma troca obrigatória.
- Princípios clássicos (as "três maneiras"): fluxo (do desenvolvimento à produção), feedback (rápido e em todas as etapas) e aprendizado contínuo (cultura de experimentação, post-mortem sem culpados).
- DevSecOps incorpora segurança ao pipeline desde o início ("shift left"): a verificação de segurança deixa de ser uma etapa final e passa a rodar a cada commit. Segurança é responsabilidade de todos, com automação e políticas como código.
- Técnicas de segurança no pipeline: SAST (análise estática do código-fonte, sem executar), DAST (testes dinâmicos contra a aplicação em execução), SCA (análise de composição de software: dependências e licenças vulneráveis), varredura de segredos (chaves e senhas no repositório), varredura de imagens de contêiner (Trivy, no laboratório L5) e SBOM (lista de componentes do software).
- Segredos nunca vão para o repositório nem para a imagem: use variáveis protegidas do CI, cofres de segredos e, no Kubernetes, Secrets (que são apenas codificados em base64, não criptografados por padrão).

- A integração, a entrega e a implantação contínuas são práticas de engenharia; DevOps é a cultura que as sustenta. Ferramenta sozinha não é DevOps.

2. Git: o modelo de dados e as três áreas

- Git é um sistema de controle de versão distribuído: cada clone tem o histórico completo. Isso permite trabalhar offline, criar branches baratas e não depender de um servidor central para consultar o histórico.
- O Git guarda um grafo de objetos: blob (conteúdo de um arquivo), tree (um diretório que aponta para blobs e trees), commit (um instantâneo: aponta para uma tree, para o commit pai, e leva autor, data e mensagem) e tag anotada. O commit é identificado por um hash (SHA-1, 40 caracteres hexadecimais). No laboratório L1, `git cat-file -p HEAD` mostra tree, parent, author e a mensagem, e `git cat-file -t` confirma commit, tree e blob.
- O Git guarda instantâneos (snapshots) e não diferenças: cada commit referencia a árvore completa, e o armazenamento compacta objetos parecidos.
- As três áreas de trabalho: diretório de trabalho (working tree, onde você edita), área de preparação (index ou staging area, o que irá no próximo commit) e repositório (os commits em .git). `git add` move do diretório de trabalho para o index; `git commit` grava o index como commit.
- `git status --short` mostra o estado em duas colunas (index e diretório de trabalho): `??` é arquivo não rastreado, `A` é arquivo novo no index, `M` é modificado e não preparado, `M` é modificado e preparado, `AA` (ou `UU`) é conflito. No L1 você vê cada estado acontecer.
- `git diff` compara diretório de trabalho com o index; `git diff --staged` (ou `--cached`) compara o index com o último commit. Depois do `git add`, o `git diff` simples fica vazio.
- `git init` cria o repositório; `git clone` copia um repositório remoto inteiro (e já configura o remoto `origin`); `git log --oneline --graph --all` desenha o grafo; `.gitignore` lista o que não deve ser rastreado (um arquivo já rastreado continua rastreado, mesmo que entre no .gitignore, até ser removido do index com `git rm --cached`).
- HEAD é o ponteiro para o commit atual (normalmente aponta para uma branch). HEAD~1 e HEAD^ são o pai do commit atual. Branch é apenas um ponteiro móvel para um commit; criar uma branch não copia arquivos.

3. Git: branches, merge, rebase, reset, revert, stash e ferramentas

- Criar e trocar de branch: `git switch -c nova` (equivale a `git checkout -b nova`); `git switch main` volta. Listar: `git branch`; apagar uma branch já integrada: `git branch -d`; apagar à força: `-D`.
- Merge fast-forward: se a branch de destino não andou, o merge apenas move o ponteiro, sem criar commit (L1, seção 3: "Fast-forward"). Merge sem fast-forward (`--no-ff`) sempre cria um commit de merge com dois pais, preservando no histórico que existiu uma branch (L1, seção 4: `git cat-file -p HEAD` mostra duas linhas `parent`).

- Conflito de merge: acontece quando as duas branches alteraram a mesma região do mesmo arquivo. O Git para, marca o arquivo com `<<<<<<<`, `=======` e `>>>>>>>`, e o status mostra `AA` (ou `UU`). Resolva editando, `git add arquivo` e `git commit` (L1, seção 5). `git merge --abort` desiste.
- Rebase: reaplica os commits da sua branch sobre outra base, criando commits novos (hashes diferentes) e deixando a história linear, sem commit de merge (L1, seção 6). Regra de ouro: nunca faça rebase de commits que já foram publicados e que outras pessoas usam, pois isso reescreve a história compartilhada. `git pull --rebase` aplica o mesmo raciocínio ao atualizar.
- `git reset` move a branch atual para outro commit. `--soft` mantém index e arquivos; `--mixed` (padrão) desfaz o index e mantém os arquivos; `--hard` descarta também as alterações nos arquivos. Os três aparecem em sequência no L1, seção 7. Reset reescreve história: não use em commits já publicados.
- `git revert` cria um novo commit que desfaz o efeito de um commit anterior, sem reescrever a história; é a forma segura de desfazer algo já publicado. Para reverter um commit de merge é preciso indicar o pai com `-m 1`.
- `git stash` guarda alterações não commitadas numa pilha e limpa o diretório de trabalho; `git stash pop` reaplica e remove da pilha; `git stash list` lista; `apply` reaplica sem remover.
- `git cherry-pick <commit>` copia um commit específico para a branch atual (L1, seção 8). `git bisect` faz busca binária no histórico para achar o commit que introduziu um defeito; com `bisect run` o teste é automático (L1, seção 9).
- `git reflog` registra para onde o HEAD andou localmente; permite recuperar commits "perdidos" por um reset --hard (L1: `git reset --hard HEAD@{1}`).
- Tags: `git tag v1.0.0` (leve) e `git tag -a v1.0.0 -m "msg"` (anotada, com autor, data e mensagem). Tags marcam versões de liberação.
- Mensagens de commit: convenção "Conventional Commits" (`feat:`, `fix:`, `chore:`, `docs:`) ajuda a gerar changelog e a decidir a versão semântica automaticamente.

4. Git remoto, fluxos de branching, pull request e hooks

- Remoto: `git remote add origin URL`, `git remote -v`, `git remote remove`. `git fetch` baixa os objetos e atualiza as branches remotas (`origin/main`) sem tocar no seu trabalho; `git pull` é `fetch` mais integração (merge, ou rebase com `--rebase`). `git push -u origin branch` envia e configura o rastreamento (L2, seções 3 e 4).
- Push rejeitado (non-fast-forward): o remoto tem commits que você não tem. A solução é `git pull --rebase` (ou merge) e então `git push` de novo; `--force` só com muita cautela, de preferência `--force-with-lease` (L2, seção 2).
- Um repositório bare (sem diretório de trabalho) é o que fica no servidor; o L2 usa um repositório bare local como "servidor" de duas pessoas.
- Git Flow: branches permanentes `main` (produção, cada commit é uma versão) e `develop` (integração); branches de apoio `feature/*` (saem de develop e voltam para develop), `release/*` (preparação de uma versão: sai de develop, volta para main e develop, recebe tag) e `hotfix/*` (correção urgente: sai de main, volta para main e develop). Indicado para versões planejadas e múltiplas versões suportadas; tem mais overhead (L2, seção 6; explicador animado do módulo).

- GitHub Flow: só `main` sempre implantável; cada mudança vai numa branch curta, abre-se um pull request, passa por revisão e CI e é mesclada. É simples e combina com entrega contínua.
- Trunk-based development: todos integram no tronco (`main`/`trunk`) com frequência, no mínimo uma vez ao dia, com branches de vida curta (horas) ou direto no tronco. Mudanças incompletas ficam escondidas por feature flags (chaves de funcionalidade), separando implantação de liberação (L2, seção 7). Exige testes automatizados fortes e é a base da integração contínua de verdade.
- Pull request (GitHub e Bitbucket) e merge request (GitLab) são a proposta de integrar uma branch em outra, com revisão de código, discussão, verificações automáticas obrigatórias (status checks) e aprovação. Proteção de branch impede push direto na main e exige revisão e CI verde.
- Estratégias de integração de um PR: merge commit (preserva tudo), squash (um único commit) e rebase and merge (linear).
- Hooks: scripts em `.git/hooks` executados em eventos. `pre-commit` roda antes do commit e, se sair com código diferente de zero, o commit é recusado (L2, seção 8). Hooks locais não são versionados nem obrigatórios; para garantir a regra, repita a verificação no CI.
- GitHub e GitLab são plataformas de hospedagem e colaboração baseadas em Git: repositórios, issues, PRs/MRs, revisão, wiki, registro de pacotes e de contêineres, e automação (GitHub Actions; GitLab CI/CD). O Git em si não depende de nenhuma delas.
- Inspeção: `git log --author`, `git blame` (quem mudou cada linha), `git shortlog -sn` (commits por autor) (L2, seção 9).

5. Integração contínua (CI)

- Integração contínua é a prática de integrar o código de todos na linha principal com frequência (várias vezes ao dia), e de cada integração disparar automaticamente build e testes. O objetivo é descobrir cedo que a integração quebrou, quando o defeito ainda é barato.
- Práticas da CI: um único repositório de código-fonte; build automatizado e auto-testável; todos fazem commit na linha principal com frequência; cada commit gera um build na máquina de integração; manter o build rápido; consertar um build quebrado é prioridade máxima; testar em ambiente parecido com o de produção.
- Um pipeline típico de CI: obter o código, restaurar dependências (com cache), análise estática e lint, compilação (build), testes unitários, testes de integração, cobertura, empacotamento do artefato e publicação em um repositório de artefatos.
- Pirâmide de testes: muitos testes unitários (rápidos e baratos), menos testes de integração e poucos testes ponta a ponta (lentos e frágeis). Testes lentos entram em estágios posteriores; a falha rápida (fail fast) economiza tempo.
- Artefato é o resultado versionado e imutável do build (JAR, imagem de contêiner, pacote). A regra "construa uma vez, implante em vários ambientes" evita que o que foi testado seja diferente do que vai para produção.
- Ferramentas de build: Maven (Java; `pom.xml`, ciclo de vida com fases validate, compile, test, package, verify, install, deploy; dependências em repositório central), Gradle,

npm, pip. Maven e Gradle gerenciam dependências, não apenas compilam. Ver a seção 15.
- Servidores e plataformas de CI: Jenkins, GitHub Actions, GitLab CI/CD, Azure Pipelines, CircleCI, Travis CI. A configuração mora no repositório ("pipeline como código").
- Um build deve ser reprodutível: mesmo commit, mesmo resultado. Fixar versões de dependências (lockfiles) e usar agentes limpos (efêmeros ou contêineres) ajuda.

6. Entrega contínua, implantação contínua e estratégias de liberação

- Entrega contínua (continuous delivery): todo commit que passa no pipeline gera um artefato que está pronto para ir a produção, mas a implantação em produção é uma decisão humana (aprovação manual). Implantação contínua (continuous deployment): vai a produção automaticamente, sem aprovação, se todos os testes passarem. Toda implantação contínua inclui entrega contínua; o inverso não vale.
- O pipeline de entrega promove o mesmo artefato por ambientes: desenvolvimento (build e testes rápidos), homologação (testes de aceitação, integração, desempenho; dados e configuração parecidos com produção) e produção (usuários reais). O que muda entre ambientes é a configuração, nunca o artefato.
- Implantação (deploy) e liberação (release) são coisas diferentes: implantar é colocar o código no ambiente; liberar é expor a funcionalidade aos usuários. Feature flags permitem implantar desligado e liberar depois.
- Estratégias de implantação: recreate (derruba tudo e sobe a nova versão: há indisponibilidade), rolling update (substitui as instâncias aos poucos, sem indisponibilidade, mas com duas versões convivendo), blue/green (dois ambientes idênticos; troca-se o tráfego de uma vez e o rollback é voltar a chave, ao custo de dobrar a infraestrutura) e canary (libera a nova versão para uma pequena parcela de usuários, observa métricas e amplia gradualmente).
- Rollback é voltar à versão anterior; roll-forward é corrigir adiante com uma nova versão. Migrações de banco devem ser compatíveis com a versão anterior (expandir e contrair) para permitir rollback.
- Gates (portões) são verificações obrigatórias entre estágios: testes, qualidade (SonarQube), segurança, aprovação manual em ambientes protegidos.
- A implantação contínua exige alta confiança nos testes, monitoramento e capacidade de reverter rápido. O explicador animado "pipeline de CI/CD" mostra o fluxo.
- Versionamento semântico: MAJOR.MINOR.PATCH. MAJOR muda quando há incompatibilidade, MINOR quando entra funcionalidade compatível e PATCH quando é correção compatível.

7. GitHub Actions

- GitHub Actions é a plataforma de automação do GitHub. Os workflows são arquivos YAML em `.github/workflows/`. Um workflow tem: `name`, `on` (eventos que o disparam: `push`, `pull_request`, `schedule` com cron, `workflow_dispatch` manual, `release`), `jobs`, e cada job tem `runs-on` (o executor, por exemplo `ubuntu-latest`) e `steps`.

- Job é um conjunto de steps que roda no mesmo executor (runner). Jobs rodam em paralelo por padrão; `needs` cria dependência e ordem: um job só começa quando os que ele "precisa" terminaram com sucesso. Se um job falha, os dependentes são pulados (L3, seções 3 e 4).
- Step usa uma action pronta (`uses: actions/checkout@v4`) ou executa um comando de shell (`run:`). Actions de terceiros devem ser fixadas por versão ou, melhor, pelo SHA do commit.
- Matriz (`strategy.matrix`) repete o job com combinações de parâmetros (versões da linguagem, sistemas operacionais), em paralelo (L3: teste em imagens Python 3.12 e 3.14).
- Condições com `if:` (por exemplo, `if: github.event_name == 'push' && github.ref == 'refs/heads/main'`) decidem se um job ou step roda; no L3, seção 6, o deploy não roda no evento `pull_request`.
- `secrets.NOME` guarda segredos criptografados e mascarados nos logs; `vars` são variáveis de configuração; `env` define variáveis de ambiente; `GITHUB_TOKEN` é um token temporário criado a cada execução, com permissões ajustáveis por `permissions:` (princípio do menor privilégio).
- `actions/cache` guarda dependências entre execuções; `actions/upload-artifact` e `download-artifact` passam arquivos entre jobs e guardam resultados. `environment:` liga um job a um ambiente (produção) que pode exigir aprovação.
- Runners: hospedados pelo GitHub (máquinas virtuais limpas e efêmeras) ou auto-hospedados (self-hosted). Código de terceiros em runner auto-hospedado público é um risco.
- No laboratório L3 o workflow foi validado com actionlint e executado localmente com a ferramenta `act` (que usa contêineres Docker para simular os runners): lint, depois testes em matriz, build e deploy com `needs`. A execução em runners hospedados reais do GitHub não foi testada (não testado).
- Um erro clássico de sintaxe é o `needs` apontar para um job que não existe ou o uso de `runs-on` errado; o actionlint pega isso (L3, seção 7).

8. GitLab CI/CD e Jenkins

- GitLab CI/CD é configurado no arquivo `.gitlab-ci.yml` na raiz do repositório. Estrutura: `stages` (ordem das etapas, por exemplo build, test, deploy), e jobs que declaram `stage`, `script` e outras chaves. Jobs do mesmo stage rodam em paralelo; o stage seguinte só começa se todos os jobs do anterior passarem.
- Chaves principais: `image` (contêiner usado), `before_script` e `after_script`, `variables`, `artifacts` (arquivos que passam para os próximos jobs), `cache`, `rules` (quando o job existe: por exemplo `if: $CI_COMMIT_BRANCH == "main"`), `only/except` (forma antiga), `needs` (cria um grafo acíclico dirigido, permitindo começar antes do fim do stage), `when: manual` (job manual), `environment`, `include` (reaproveitar YAML), `extends` e âncoras YAML.
- Variáveis predefinidas: `CI_COMMIT_SHA`, `CI_COMMIT_BRANCH`, `CI_PIPELINE_ID`, `CI_REGISTRY_IMAGE`. Variáveis protegidas e mascaradas guardam segredos, e podem ser limitadas a branches protegidas.

- O executor do GitLab é o GitLab Runner (shell, Docker, Kubernetes). No laboratório L4, o pipeline foi executado localmente com `gitlab-ci-local`, mostrando os stages, o `needs` e as `rules` que fazem os jobs de deploy só existirem na branch main. O comportamento de `workflow:rules` em um servidor GitLab real não foi testado.
- Jenkins é um servidor de automação open source, escrito em Java, muito usado em ambientes corporativos. A arquitetura é controlador (controller, antigo master) e agentes (agents). Tem enorme ecossistema de plugins, o que é força e também fonte de manutenção e de falhas de segurança.
- O pipeline do Jenkins é definido num `Jenkinsfile` versionado com o código. Há dois estilos: declarativo (`pipeline { agent any; stages { stage('Build') { steps { sh 'mvn package' } } } }`), mais estruturado e recomendado, e scripted (Groovy puro). Um Jenkinsfile não foi executado neste módulo (não testado).
- Comparação: GitHub Actions e GitLab CI são integrados à plataforma Git e usam YAML; Jenkins é independente, mais flexível e mais trabalhoso de operar (você mantém o servidor e os plugins). Todos suportam pipeline como código, agentes efêmeros em contêineres e gatilhos por webhook.
- Webhook é uma chamada HTTP que o servidor Git faz quando algo acontece (push, PR), usada para disparar o pipeline. Alternativa menos eficiente: polling.
- Ferramentas correlatas citadas em provas: SonarQube (qualidade e dívida técnica, análise estática), Nexus e Artifactory (repositórios de artefatos), Trivy (varredura), Argo CD e Flux (GitOps), Azure DevOps Pipelines, Bitbucket Pipelines, CircleCI.

9. DevSecOps, qualidade e segurança no pipeline

- Princípio "shift left": antecipar verificação de segurança e qualidade para as primeiras etapas (IDE, commit, pull request) em vez de deixá-las para o final.
- SAST analisa o código-fonte ou o bytecode sem executar o programa; acha, por exemplo, injeção de SQL e uso de funções inseguras. DAST testa a aplicação em execução, de fora (como um atacante), achando falhas de configuração e de autenticação. IAST instrumenta a aplicação durante os testes. SCA inventaria dependências de terceiros e avisa de vulnerabilidades conhecidas (CVE) e licenças. Varredura de segredos procura chaves, tokens e senhas em commits.
- Varredura de imagem de contêiner: o Trivy, no L5, analisou uma imagem e contou vulnerabilidades por severidade (LOW, MEDIUM, HIGH). Imagens menores (alpine, distroless, multi-stage) têm menos pacotes e, portanto, menos superfície de ataque.
- SBOM (Software Bill of Materials) é o inventário dos componentes de um software; assinatura de artefatos e de imagens garante a origem (cadeia de suprimentos).
- Qualidade: cobertura de testes, complexidade ciclomática, duplicação de código e dívida técnica, medidos por ferramentas como o SonarQube; "quality gates" reprovam o build se os limites forem violados.
- Princípios: menor privilégio para tokens e contas de serviço, segregação de funções (quem aprova não é quem implanta, quando exigido), trilha de auditoria (todo deploy rastreável a um commit e a um responsável) e política como código.
- Em contêineres: não rodar como root (L5, seção 6: usuário 10001), sistema de arquivos somente leitura, remover capacidades (capabilities) desnecessárias, não usar `--`

`privileged` (L5, seção 10), fixar versões de imagem e não colocar segredos em camadas da imagem.
- Em Kubernetes: RBAC, Network Policies, Pod Security Standards, Secrets e limites de recursos.

10. Docker: contêineres, imagens, Dockerfile, camadas e cache

- Contêiner é um processo (ou grupo de processos) isolado do restante do sistema, que compartilha o kernel do hospedeiro. Isso o torna muito mais leve que uma máquina virtual, que tem seu próprio sistema operacional completo sobre um hipervisor. No L5, `uname -r` no host e dentro de um contêiner alpine mostram o mesmo kernel.
- O isolamento vem de recursos do kernel Linux: namespaces (isolam o que o processo enxerga: PID, rede, montagens, usuários, hostname) e cgroups (limitam e medem o uso de CPU, memória e E/S). Docker usa também sistemas de arquivos em camadas (overlay).
- Arquitetura: o cliente `docker` conversa com o daemon (`dockerd`) por uma API REST, normalmente por um socket UNIX (L5, seção 1). Abaixo do daemon está o containerd e o runc, que cria o contêiner. Registry (Docker Hub, GitHub Container Registry, GitLab Registry, registro privado) armazena imagens.
- Imagem é um modelo somente leitura, composto de camadas imutáveis empilhadas; contêiner é uma instância em execução da imagem, com uma camada superior gravável. Ao apagar o contêiner, a camada gravável se perde, por isso dados persistentes vão em volumes.
- Dockerfile é a receita da imagem. Instruções: `FROM` (imagem base), `WORKDIR`, `COPY` e `ADD`, `RUN` (executa durante o build e cria camada), `ENV`, `ARG` (variável só do build), `EXPOSE` (documenta a porta, não publica), `USER`, `HEALTHCHECK`, `ENTRYPOINT` (o executável) e `CMD` (argumentos padrão, substituíveis). `docker build -t nome:tag .` constrói; `docker run` cria e inicia um contêiner (`-d` em segundo plano, `-p 8080:80` publica a porta, `--name`, `--rm`, `-e`, `-v`).
- Cache de camadas: cada instrução gera uma camada e o Docker reaproveita a camada se a instrução e tudo o que veio antes forem idênticos. Quando uma camada muda, todas as seguintes são refeitas. Por isso o Dockerfile deve copiar primeiro o que muda pouco (requirements.txt ou pom.xml), instalar dependências, e só depois copiar o código (L5, seções 2 e 3: na ordem ruim o `pip install` roda a cada mudança; na ordem boa aparece `CACHED`). O explicador animado "camadas e cache" mostra isso.
- `.dockerignore` mantém fora do contexto de build o que não deve entrar (`.git`, `node_modules`, segredos): no L5, seção 4, o contexto enviado ao daemon caiu de cerca de 21 MB para alguns kB e o arquivo de segredo deixou de ir.
- `docker commit` cria imagem a partir de um contêiner modificado, mas não é reprodutível nem auditável; o correto é um Dockerfile versionado (L5, seção 7).
- Comandos de dia a dia: `docker ps` (`-a` inclui parados), `docker logs`, `docker exec -it`, `docker stop` e `docker rm`, `docker images`, `docker rmi`, `docker inspect`, `docker pull` e `docker push`.
- Tags: `nome:1.2.3` é um nome para um hash de imagem; `latest` é só uma tag convencional (não significa "mais recente") e deve ser evitada em produção, pois não é

reprodutível. Fixar por versão (ou digest) garante o mesmo resultado.

11. Docker: multi-stage, registry, volumes, redes, Compose e segurança

- Build multi-stage: usa vários `FROM` em um Dockerfile; os primeiros estágios têm compilador e ferramentas, o último copia só o resultado (`COPY --from=build`). No L5, seção 5, a imagem Go final tem cerca de 2 MB contra 412 MB da imagem sem multi-stage. Menor imagem é mais rápida de distribuir e tem menos superfície de ataque.
- Registry: `docker tag app localhost:5000/app:1.0` dá um novo nome e `docker push` envia. No L5, seção 8, um registry local (a imagem `registry:2`) recebeu um push e depois um pull.
- Volumes persistem dados fora do ciclo de vida do contêiner: volume nomeado (gerenciado pelo Docker), bind mount (pasta do hospedeiro) e tmpfs (memória). No L5, seção 9 e L6, o contador continua depois de recriar o contêiner porque os dados estão em volume.
- Redes: `bridge` é o padrão, `host` usa a rede do hospedeiro, `none` sem rede. Em uma rede definida pelo usuário, os contêineres se encontram por nome (DNS embutido); na bridge padrão isso não funciona. `-p` publica uma porta do contêiner no hospedeiro.
- Docker Compose descreve uma aplicação com vários contêineres em um arquivo YAML (`compose.yaml`): `services`, `image` ou `build`, `ports`, `environment`, `volumes`, `networks`, `depends_on`. Comandos: `docker compose up -d`, `down` (`-v` apaga também os volumes), `ps`, `logs`, `exec`, `config`.
- `depends_on` só garante a ordem de início; para esperar o serviço estar pronto use `condition: service_healthy` com um `healthcheck` (L6, seção 2: o web só sobe quando o Redis está saudável). Arquivos de override (`compose.override.yaml`, `-f`) e variáveis de ambiente adaptam o mesmo arquivo a cada ambiente (L6, seção 4).
- Compose é para desenvolvimento, testes e implantações simples em um único hospedeiro; para vários hospedeiros, escala e autocura, usa-se orquestrador (Kubernetes; o Docker Swarm existe mas não foi testado aqui).
- Segurança de contêiner (L5, seções 6 e 10): usuário não-root (`USER`), `--read-only`, `--cap-drop ALL`, `--security-opt no-new-privileges`, limites de memória e CPU (`--memory`, `--cpus`), nunca `--privileged` nem montar o socket do Docker em um contêiner. A varredura com Trivy (L5, seção 11) acha vulnerabilidades conhecidas.
- `HEALTHCHECK` na imagem marca o contêiner como `healthy` ou `unhealthy` (L5, seção 6); o Kubernetes usa probes próprias.
- Boas práticas: uma preocupação por contêiner, imagens enxutas, logs na saída padrão (stdout/stderr, que o `docker logs` lê), configuração por variáveis de ambiente e contêineres descartáveis (sem estado local).

12. Kubernetes

- Kubernetes (K8s) é um orquestrador de contêineres: agenda, escala, conecta, atualiza e recupera contêineres em um cluster de máquinas. O estado desejado é declarado em YAML e controladores trabalham continuamente para fazer o estado real convergir para ele (laço de reconciliação).

- Arquitetura: o plano de controle (control plane) tem o `kube-apiserver` (porta de entrada da API), o `etcd` (armazém chave-valor do estado do cluster), o `kube-scheduler` (escolhe o nó de cada Pod) e o `kube-controller-manager` (executa os controladores). Cada nó de trabalho (worker) tem o `kubelet` (garante que os contêineres dos Pods estejam rodando), o `kube-proxy` (regras de rede dos Services) e um runtime de contêineres (containerd, CRI-O). No L8 um cluster kind de dois nós mostra o runtime `containerd`.
- Pod é a menor unidade implantável: um ou mais contêineres que compartilham rede (mesmo IP) e volumes. Pods são efêmeros; não se criam Pods "soltos" em produção.
- ReplicaSet mantém um número de réplicas de um Pod. Deployment gerencia ReplicaSets e permite atualização gradual (rolling update) e rollback (`kubectl rollout status`, `history`, `undo`; `kubectl scale`). No L8, a troca da imagem criou um novo ReplicaSet e o `rollout undo` voltou ao anterior.
- Service dá um IP e um nome DNS estáveis a um conjunto de Pods selecionados por labels. Tipos: ClusterIP (interno, padrão), NodePort (porta em cada nó), LoadBalancer (balanceador do provedor de nuvem) e ExternalName. Ingress expõe rotas HTTP/HTTPS de fora para Services (exige um controlador de ingress).
- Um Service `kind: Service` não cria Deployment algum: ele só seleciona Pods existentes (L8, seção 9, questão do BCB: `kubectl apply` do manifesto de Service não cria Pods).
- Outros objetos: StatefulSet (identidade estável e armazenamento para aplicações com estado), DaemonSet (um Pod por nó, como agentes de log e monitoramento; no L8 é preciso uma toleration para rodar também no nó do plano de controle), Job e CronJob (execução única e agendada), ConfigMap (configuração) e Secret (dados sensíveis, em base64), PersistentVolume e PersistentVolumeClaim (armazenamento), Namespace (separação lógica: dev, hom, prod), ServiceAccount, Role e RoleBinding (RBAC).
- Probes: liveness (reinicia o contêiner se falhar), readiness (retira o Pod dos endpoints do Service se falhar, sem reiniciar) e startup. No L8, seção 6, um readiness que falha deixa o Pod fora do tráfego. Requests e limits definem CPU e memória reservados e máximos; HPA (Horizontal Pod Autoscaler) escala réplicas por métrica.
- Hooks de ciclo de vida: `postStart` e `preStop`; o preStop roda antes do SIGTERM e dá tempo de encerrar com graça (L8, seção 8).
- `kubectl`: `apply -f`, `get`, `describe`, `logs`, `exec`, `delete`, `port-forward`, `rollout`, `scale`, `create ... --dry-run=client -o yaml` (gera um manifesto). `kubectl apply` é declarativo; `kubectl create` é imperativo.
- Distribuições e ferramentas: kind e minikube (local), k3s, Rancher (gerenciamento de clusters; não testado), Helm (gerenciador de pacotes: charts; não testado), serviços gerenciados (EKS, AKS, GKE). Limite do sandbox: os Services com balanceamento entre vários Pods não puderam ser testados no ambiente dos laboratórios, porque o kernel não tem a extensão de iptables que o kube-proxy usa (L8, seção 3b).

13. Infraestrutura como código e gerenciamento de configuração

- Infraestrutura como código (IaC) é descrever e provisionar infraestrutura (redes, máquinas, bancos, permissões) em arquivos de código versionados, revisados e

executados por ferramentas, em vez de cliques manuais em consoles. Ganhos: reprodutibilidade, rastreabilidade (Git), revisão por PR, ambientes iguais e recuperação rápida; reduz "snowflake servers" e deriva de configuração.
- Declarativa (descreve o estado final desejado: Terraform, OpenTofu, CloudFormation, Kubernetes) e imperativa/procedural (descreve os passos: scripts, Ansible em sua forma de tarefas ordenadas). Mutável (altera servidores no lugar) e imutável (substitui o recurso por um novo com a configuração nova).
- Provisionamento (criar a infraestrutura: Terraform) e gerenciamento de configuração (configurar o que está dentro dos servidores: Ansible, Puppet, Chef, Salt) são complementares, embora haja sobreposição.
- Terraform (e o fork OpenTofu, usado nos laboratórios) usa HCL e o fluxo `init` (baixa providers e módulos), `fmt`, `validate`, `plan` (mostra o que será criado, alterado ou destruído, sem aplicar), `apply` (executa) e `destroy`. O arquivo de estado (`terraform.tfstate`) mapeia o código aos recursos reais; deve ficar em um backend remoto, com bloqueio e criptografia, e nunca no Git, pois pode conter segredos. No L7: `Plan: 3 to add`, depois `Apply complete: 3 added`.
- Idempotência: aplicar a mesma configuração várias vezes leva ao mesmo estado e a segunda execução não faz nada (L7, seção 4: "No changes"; Ansible: `changed=3` na primeira e `changed=0` na segunda). Mudar um atributo que não pode ser alterado no lugar força substituição (destroy e create), como o plan mostrou no L7, seção 5.
- Drift (deriva) é a diferença entre o código e o mundo real, causada por mudança manual. O plan detecta e o apply corrige (L7, seção 6). Variáveis (`variable`), saídas (`output`), módulos (reuso), `count` e `for_each`, e validações de variável permitem reaproveitar o código em ambientes (L7, seção 7).
- Ansible é agentless (sem agente: usa SSH), declarativo na intenção mas procedural na ordem, e escreve em YAML: inventário (hosts), playbooks, tasks, módulos (idempotentes), roles e handlers. `--check --diff` simula (L7, seção 9).
- Puppet e Chef usam agente e modelo "pull" (o nó busca a configuração); Ansible usa "push" por padrão. Terraform e o Ansible podem trabalhar juntos: um provisiona, o outro configura.
- Segurança em IaC: segredos fora do código (cofres), varredura estática dos arquivos (tfsec, Checkov), revisão por PR e execução do plan no CI. Recursos de nuvem reais (sub-redes, balanceadores, VMs) não foram criados nos laboratórios (não testado); o L7 usou provedores locais.
- GitOps: o repositório Git é a fonte da verdade do estado desejado; um agente (Argo CD, Flux) dentro do cluster compara o repositório com o real e converge (modelo pull). Isso une IaC, Git e PR como mecanismo de mudança.

14. Observabilidade e monitoramento

- Monitoramento acompanha métricas conhecidas e alerta quando algo sai do normal ("o sistema está bem?"); observabilidade é a capacidade de entender o estado interno do sistema a partir das saídas que ele emite, inclusive para perguntas que você não previu ("por que está lento?").
- Os três pilares da telemetria: métricas (números agregados ao longo do tempo: taxa de requisições, erros, latência, uso de CPU), logs (registros de eventos, de preferência

estruturados em JSON, que podem ser consultados e filtrados) e traces (o caminho de uma requisição por vários serviços, composto por spans ligados por um trace_id; no L9, seção 7, os spans de uma requisição compartilham o mesmo trace_id).
- Instrumentação é o código (ou agente) que gera a telemetria. OpenTelemetry (OTel) é o padrão aberto de instrumentação e coleta para métricas, logs e traces, independente de fornecedor. Correlação liga métricas, logs e traces pelo identificador comum.
- Tipos de métrica no Prometheus: counter (só aumenta, e zera ao reiniciar; use `rate()`), gauge (sobe e desce, como uso de memória ou tamanho de fila), histogram (distribuição em buckets, permite `histogram_quantile` para percentis como o p95) e summary. No L9, seção 6, o counter nunca diminui e o gauge oscila.
- Prometheus: banco de dados de séries temporais com modelo pull (ele "raspa", ou scrape, o endpoint `/metrics` dos alvos), linguagem de consulta PromQL, regras de alerta e o Alertmanager (agrupa, silencia e roteia alertas). No L9, o Prometheus coletou a aplicação e uma regra de taxa de erros 500 disparou o alerta (seções 3 a 5).
- Grafana é a plataforma de visualização (painéis e dashboards) que consulta várias fontes (Prometheus, Loki, Elasticsearch). A porta padrão é 3000. Foi só lida a configuração padrão da imagem; o servidor Grafana não foi testado em execução (não testado) e o L9 verifica apenas os padrões de escuta documentados (seção 8).
- Pilha ELK/Elastic: Elasticsearch (armazenamento e busca), Logstash (ingestão e transformação), Kibana (visualização), com Beats (coletores leves). Loki guarda logs indexando só rótulos. Zabbix é uma ferramenta de monitoramento tradicional (agentes, SNMP, templates), diferente do modelo do Prometheus. Nenhum desses foi executado nos laboratórios (não testado), exceto o Prometheus.
- SLI (indicador do nível de serviço: o que se mede, como a proporção de requisições bem-sucedidas), SLO (objetivo: o alvo, como 99,9% em 30 dias) e SLA (acordo com consequências contratuais). O orçamento de erros (error budget) é o que resta entre 100% e o SLO e orienta quanto risco se pode correr nas mudanças.
- Sinais de ouro (Google SRE): latência, tráfego, erros e saturação. Método RED (rate, errors, duration) para serviços e USE (utilization, saturation, errors) para recursos.
- Bom alerta é acionável e baseado em sintoma (impacto no usuário), não em causa; evita fadiga de alertas. Runbooks, on-call e post-mortem sem culpados fecham o ciclo.

15. Build, artefatos e versionamento: Maven e repositórios

- Maven é uma ferramenta de build e gerenciamento de dependências de projetos Java, baseada em convenção sobre configuração. O arquivo `pom.xml` descreve o projeto: coordenadas `groupId`, `artifactId` e `version`, dependências, plugins e propriedades.
- Ciclo de vida padrão do Maven (o que se executa em `mvn <fase>`): validate, compile, test, package, verify, install (instala no repositório local, em `~/.m2`) e deploy (publica no repositório remoto). Executar uma fase executa todas as anteriores. `clean` é outro ciclo, que apaga `target/`.
- Repositório de artefatos (Nexus, Artifactory, GitHub Packages, GitLab Package Registry) guarda JARs, imagens e pacotes versionados; versões de lançamento são imutáveis e versões de desenvolvimento usam o sufixo `-SNAPSHOT` no Maven.
- Gradle é a alternativa (scripts Groovy ou Kotlin, build incremental e cache). npm, pip e outros têm lockfiles para travar versões.

- Boas práticas de versão: versionamento semântico; o número da versão sai da tag do Git; o artefato carrega o hash do commit para rastreabilidade; imagem Docker tem tag fixa (versão e hash), não apenas `latest`.
- Builds reprodutíveis e cache de dependências aceleram o CI. Dependências devem ser verificadas (SCA) a cada build.
- Migrações de banco de dados também são código: ferramentas como Flyway e Liquibase versionam o esquema, e o pipeline as aplica.

16. Ambientes, SRE, entrega confiável e fechamento do módulo

- Ambientes: desenvolvimento (cada dev e integração rápida, dados fictícios), testes (automatizados), homologação (também chamado de staging ou pré-produção: validação por usuários e testes de aceitação, o mais parecido com produção) e produção. Cada ambiente tem sua configuração e seus segredos; o artefato é o mesmo.
- Paridade entre ambientes reduz o "na minha máquina funciona". Os 12 fatores (12-factor app) recomendam configuração por variáveis de ambiente, processos sem estado, logs como fluxo de eventos e dependências declaradas.
- Ambientes efêmeros (por branch ou por PR) com contêineres e IaC permitem testar mudanças isoladas e descartar depois.
- SRE (Site Reliability Engineering) aplica engenharia de software às operações: define SLOs, usa orçamento de erros, automatiza o trabalho repetitivo (toil) e pratica post-mortem sem culpados. Relaciona-se a DevOps: "SRE implementa DevOps".
- Gestão de mudanças moderna: lotes pequenos, automação, revisão por pares e reversibilidade são mais seguros do que grandes liberações raras e aprovadas por comitê.
- Cadeia completa para lembrar: commit no Git, PR com revisão, CI (build e testes), artefato versionado (imagem), IaC provisiona o ambiente, deploy no Kubernetes (rolling, canary), observabilidade (métricas, logs, traces) e feedback para o próximo ciclo.
- Onde cada ferramenta se encaixa: Git (código), GitHub/GitLab (colaboração), Actions/GitLab CI/Jenkins (automação), Maven (build), Docker (empacotamento), Compose (várias peças em um hospedeiro), Kubernetes (orquestração), Terraform/Ansible (infraestrutura e configuração), Prometheus/Grafana/ELK (observabilidade), SonarQube/Trivy (qualidade e segurança).
- Fora do escopo, que apareceu em provas e foi deixado de fora deste módulo: serverless e microsserviços como tema próprio (questões do BCB 84 a 86, do TCU 152 e da FUNPRESP 64 pertencem ao tema de arquitetura).